During our recent migration in Armanino from Databricks to Microsoft Fabric, our engineering team identified key operational friction points related to capacity utilization, performance optimization, workspace deployment, and engine behavior. By analyzing these architectural challenges, we established clear engineering guidelines that eliminate unexpected cost overruns, optimize processing speeds, and stabilize reporting performance.

Even a logically correct solution is incorrect if the cost is too high so the development phase should involve proper architectural decisions for cost optimization.

Below is a detailed breakdown of the challenges encountered, root causes, financial/operational impacts, and implemented solutions.

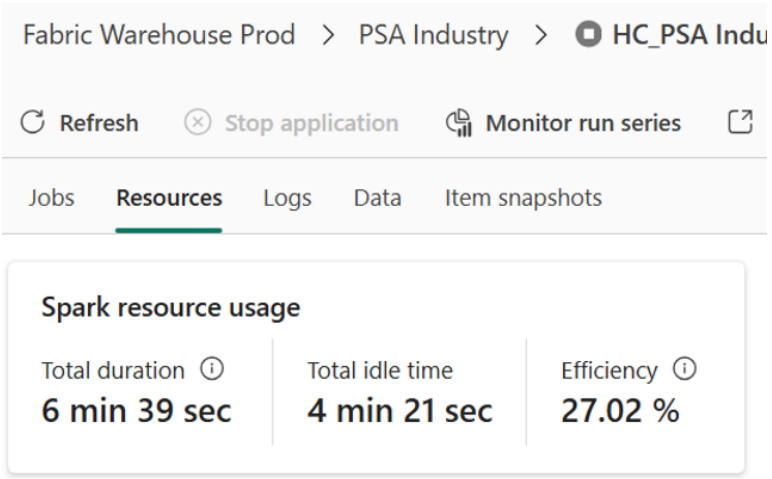
Technical Challenges & Standard Solutions

1. Capacity Quota Exhaustion from Small Workloads

- **The Challenge:** Running PySpark notebooks for low-to-medium data processing tasks quickly exhausted our daily Fabric Capacity Units (CU) / Capacity Quota. This resulted in capacity throttling, leaving insufficient compute for Power BI reports and executive dashboards, while exposing the organization to potential compute overage costs.
- **Root Cause:** A Spark cluster incurs fixed startup overhead and compute costs regardless of data size. Spinning up multi-node Spark clusters for small datasets (e.g., thousands of rows) is compute-inefficient and wasteful.
- **The Fix & Engineering Rule:**
    - **Rule:** If the target dataset contains **fewer than 100,000 rows (or < 100 MB)**, **PySpark must not be used**.
    - **Action:** Route these workloads through standard **Fabric Data Factory Pipelines** or **Single-Core Python** routines within lightweight execution environments. Reserve PySpark exclusively for high-volume, distributed data processing.

2. Low PySpark Efficiency & Unutilized Parallelism

- **The Challenge:** Several PySpark notebooks exhibited low processing efficiency (< 60%), consuming high CUs while delivering slow execution times.
- **Root Cause:** In our implementation, notebooks pulled data sequentially (one chunk at a time) from API endpoints. Because the underlying logic relied on single-threaded REST calls, Spark's distributed architecture remained idle while waiting on sequential execution.
- **The Fix:**
    - **Rule:** All Fabric PySpark jobs must maintain a target execution efficiency score of **≥ 60%** in performance diagnostics.This can be checked in the resources tab of the notebook monitor.Below screenshot is example of poor notebook efficiency.



- **Action:** Redesigned the ingestion logic to execute **parallelized API calls** using concurrent thread pools within PySpark. This fully saturates Spark workers, decreases job execution times, and dramatically lowers compute consumption.

3. Lakehouse vs. Warehouse Architectural Strategy

- **The Challenge:** Developers frequently opted for Fabric Data Warehousing due to familiarity with standard T-SQL. However, Warehouse instances resulted in significantly higher compute costs compared to Lakehouse setups.
- **Root Cause:** Fabric Data Warehouses use a transactional SQL compute engine designed for complex multi-table joins and traditional warehousing workloads, making them more resource-intensive than Lakehouse operations.

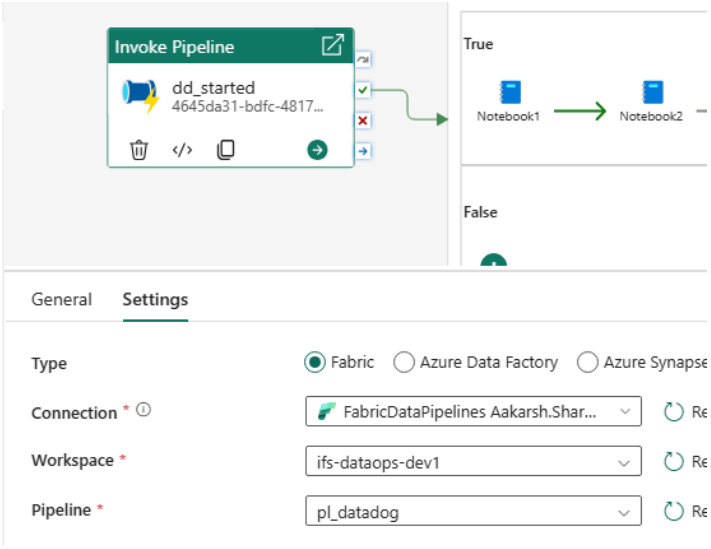
  Fabric Warehouses utilize a dedicated distributed T-SQL MPP (Massively Parallel Processing) compute engine designed for heavy relational transactional integrity (ACID) and multi-table DML (INSERT/UPDATE/DELETE). While powerful, keeping this SQL engine active for routine ETL/ELT leads to significantly higher Capacity Unit (CU) consumption. Also, lakehouse is compatible with medallion architecture, supports all data types(structured,semi-structured and unstructured) and has compression enabled for better storage performance.

- **The Fix & Architectural Guidance:**
  - **Primary Recommendation (Microsoft Best Practice):** Adopt **Lakehouse** as the default storage and transformation architecture to ptimize costs.
  - **Comparison & Decision Matrix:**

Dimension	**Fabric Lakehouse (Recommended)**	**Fabric Warehouse**

Primary Engine	Apache Spark / Delta Engine	T-SQL Compute Engine
Developer Skillset	Python, PySpark, Spark SQL	T-SQL / Traditional DBA
Cost Profile	**Lower** (Direct Delta Lake operations)	**Higher** (Dedicated SQL compute)
Best Used For	Data engineering, ETL/ELT, large-scale processing, Direct Lake reporting	Traditional relational data warehousing, strict ACID enterprise SQL requirements

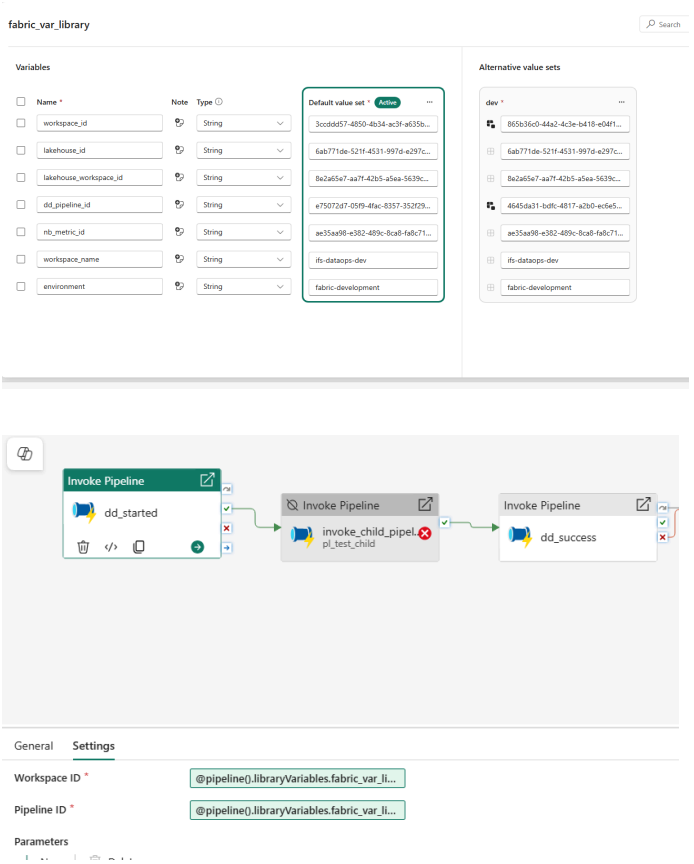
4. Hardcoded Environment Variables in Pipeline Deployment

- **The Challenge:** Migrating Fabric Data Factory pipelines across deployment environments (Dev → Test → Prod) required manual updates to workspace IDs, Lakehouse references, and connection parameters via the UI, creating risks of misconfiguration and deployment delays.
  - 

Static values in UI

- **Root Cause:** Pipeline activities contained hardcoded UI bindings rather than dynamic parameters. Furthermore, requiring dedicated DevOps resource code for standard deployments was unsustainable across all project teams.
- **The Fix:**
  - **Action:** Implemented dynamic environment configuration by utilizing **Library Variables**

- **Result:** All target workspace, Lakehouse, and connection parameters are dynamically resolved at runtime based on the target environment, enabling standard CI/CD pipelines to execute without manual intervention or specialized DevOps coding.



Library Variables and Dynamic selection in UI

5. SQL End Point Delay in Lakehouse

- **The Challenge:** When querying Lakehouse Delta tables to populate downstream analytical tables, target reports frequently displayed outdated snapshots of data—even after successful refresh cycles.
- **Root Cause:** SQL End point delay is a know bug in Microsoft Fabric. Due to it SQL Endpoint of lakehouse can show old snapshot even though lakehouse has updated data
- **The Fix:**
  - **Action:** Execution of metadata refresh scripts immediately following data load operations like discussed below

    https://uselessai.in/how-to-refresh-fabric-sql-endpoint-to-have-latest-data-from-lakehouse-tables-ddef23d016a9

  - **Result:** Guarantees Data synchronization in sql endpoint.

6. Semantic Model Failures in Direct Lake Mode

- **The Challenge:** Utilizing Power BI Direct Lake mode on underlying warehouse/lakehouse tables resulted in fatal reporting errors ("missing data (parquet) files in OneLake"), causing executive dashboards to crash.
- **Root Cause:** Schema modifications (such as dropping and recreating tables or changing column types) alter the underlying parquet file paths in OneLake. Direct Lake relies on direct metadata pointers to these parquet files; when schemas change, pointers break before Power BI can recalculate the model framing.
- **The Fix:**
    - Shifted critical, frequently modified tables to **Import Mode** with scheduled automated refreshes to eliminate report crashes but this requires scheduled/triggered refreshes of datamart.

7. Restrictions on Power Query & advanced data modeling feature in Direct Lake Mode

The Challenge: Severe restrictions on advanced data modeling features and data types block standard enterprise reporting patterns.

Root Cause: Direct Lake reads directly from delta parquet files into memory using the VertiPaq engine structure, which lacks support for complex Delta column types (like binary and GUID), calculated tables referencing Direct Lake columns, and specific user hierarchies. Furthermore, when complex DAX measures or unsupported structures trigger a fallback to DirectQuery or fail outright, development options become heavily restricted compared to traditional Import mode.

The Fix: Standardized warehouse data types to strings or numerics prior to model creation, refactored reports to use standard relationships instead of unsupported hierarchies, and limited advanced transformations to upstream Spark/SQL pipelines.

8.Data Governance and Security Enforcement Constraints

The Challenge: Applying native database-level security policies (like row-level security in Lakehouses or Data Warehouses) breaks Direct Lake routing, leading to query failures or unexpected performance drops.

Root Cause: Direct Lake bypasses the SQL analytics endpoint to read Parquet files natively from storage using the semantic model user's credentials. If security permissions, Object-Level Security (OLS), or Row-Level Security (RLS) are defined at the underlying warehouse/lakehouse level rather than directly inside the Power BI semantic model, Direct

Lake cannot read the files directly. The engine either blocks the connection outright or silently reverts to DirectQuery mode to enforce the database engine's security rules.

The Fix:

Migrated security logic from the underlying storage layer directly into the Power BI semantic model using DAX-based RLS, ensuring users maintain direct storage-level read access without triggering database security fallbacks.

9. Memory Paging and SKU Guardrails

The Challenge: Severe performance bottlenecks and slow query execution occur when large semantic models exceed the physical RAM limits allocated by the underlying Fabric capacity SKU.

Root Cause: Unlike traditional Import mode, which throws an out-of-memory error when data exceeds capacity limits, Direct Lake handles large datasets by dynamically paging column data in and out of memory from OneLake storage. If a dataset exceeds capacity memory thresholds, frequent disk-to-RAM swapping stalls report rendering and degrades query response times across all concurrent dashboards.

The Fix: Upgraded the Fabric capacity SKU to a higher tier to increase dedicated RAM limits, dropped unused columns from the source delta tables to reduce the in-memory footprint, and partitioned large fact tables to optimize memory usage.